Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report

on

"Hospital Management System"

[Code No.: COMP 207]

(For partial fulfillment of Year II / Semester II in Computer Science)

Submitted by

Aayush Aacharya (Roll no. 1)

Aayush Adhikari (Roll no. 2)

Aarjan Adhikari (Roll no. 3)

Krish Ghimire (Roll no. 24)

Submitted to

Dr. Rabindra Bista

Department of Computer Science and Engineering

July 26, 2026

Bona fide Certificate

This project work on

"Hospital Management System"

is the bona fide work of

Aayush Aacharya (Roll no. 1), Aayush Adhikari (Roll no. 2),
Aarjan Adhikari (Roll no. 3), Krish Ghimire (Roll no. 24)

who carried out the project work under my supervision.

Project Supervisor

Dr. Rabindra Bista
Department of Computer Science and Engineering

Acknowledgements

To everyone who contributed to the success of our project, we extend our heartfelt appreciation. We are thankful for the collective efforts and support from all those who contributed, directly or indirectly, to the successful completion of our project. Their encouragement has been invaluable, and we truly appreciate the collaborative spirit that made this endeavour possible.

A sincere gratitude goes to Dr. Rabindra Bista, our project supervisor. His unwavering support, guidance, and constructive feedback have been instrumental in steering our project towards excellence.

A special thanks to the Department of Computer Science and Engineering (DoCSE) for providing an enriching academic environment and the necessary resources that played a crucial role in the accomplishment of our team project.

Abstract

Hospital operations often become inefficient, and patient care is affected, because many hospitals rely on manual processes or disconnected management tools. This project developed a full-stack **Hospital Management System (HMS)** that unifies core clinical and administrative operations on a single digital platform. The system was implemented using a layered client–server architecture to improve scalability, maintainability, and security. The backend was developed using Node.js, Express, and the Sequelize ORM over a MySQL relational database, while a responsive web interface was built with React and Tailwind CSS to serve five distinct roles—patient, doctor, pharmacist, lab assistant, and administrator. Secure authentication was implemented using JSON Web Tokens (JWT) with refresh-token rotation, together with role-based access control, password reset, and email verification. The completed system provides features such as appointment scheduling with doctor availability, prescriptions, laboratory-test workflows, referrals, billing and invoicing, a location-aware pharmacy inventory, and multi-channel notifications delivered in-app and by email. In conclusion, the HMS successfully delivers a secure, scalable, and user-friendly hospital-management solution, and future enhancements are recommended, including database migrations, an inpatient/ward module, and predictive analytics.

Keywords: Hospital Management System, React, Node.js, Express, MySQL, Sequelize, JWT, Role-Based Access Control, RESTful API.

Contents

List of Figures

Abbreviations

HMS	Hospital Management System.
REST	Representational State Transfer.
API	Application Programming Interface.
JWT	JSON Web Token.
ORM	Object–Relational Mapping.
RBAC	Role-Based Access Control.
SPA	Single Page Application.
CRUD	Create, Read, Update, Delete.
SMTP	Simple Mail Transfer Protocol.
VS Code	Visual Studio Code.

Chapter 1

Introduction

The Hospital Management System (HMS) is a role-based web platform developed to connect the different people involved in patient care—patients, doctors, pharmacists, lab assistants, and administrators—through a single digital system. It provides a web interface where patients can register, book appointments, view prescriptions and lab results, and settle bills, while hospital staff manage schedules, clinical records, pharmacy stock, and billing. The system focuses on making everyday hospital tasks simple and reliable while keeping medical data secure. By using modern technologies, it supports consistent records and timely notifications, helping to reduce the common problems found in manual and disconnected hospital systems.

1.1 Background

The rapid growth of digital technology has changed how hospitals manage information. Patients increasingly expect to book appointments and view records online, while hospitals adopt web-based systems to manage scheduling, prescriptions, laboratory work, and billing, with modern solutions focusing on user-friendly design, reliable records, and secure access.

However, many existing systems have limitations. Some focus mainly on record-keeping while offering little role separation, and others are too complex or costly for small and medium-sized institutions. In many cases, hospitals rely on separate tools for appointments, pharmacy, and billing, which increases complexity and reduces efficiency. Sensitive medical data also demands strong authentication and access control that older systems often lack.

These issues highlight the need for a more balanced and integrated system. The HMS addresses this by combining appointment scheduling, clinical records, pharmacy inventory, billing, and notifications within one secure, role-based platform, improving interaction between patients and staff and increasing overall efficiency.

1.2 Objectives

The main objectives achieved during the development of the HMS were:

- Developed a web-based platform that enabled patients to register, book, and manage appointments efficiently.
- Created role-specific dashboards that allowed doctors, pharmacists, lab assistants, and administrators to manage their respective operations.
- Implemented core clinical and administrative workflows—prescriptions, laboratory tests, referrals, billing, and a location-aware pharmacy inventory.
- Ensured secure user authentication and role-based access control using JWT with refresh-token rotation, password reset, and email verification.
- Provided multi-channel notifications (in-app and email) so users stay informed without needing to log in.

1.3 Motivation and Significance

The motivation behind this project was to gain hands-on experience building a real-world, full-stack web application while addressing the practical limitations of manual and fragmented hospital systems. Most modern institutions depend on digital systems to manage appointments, records, and billing, and recent solutions emphasise simple interfaces, reliable data, and secure access using modern technologies.

The significance of the HMS lies in its ability to improve operational efficiency, reduce manual errors, and offer secure, role-based access to clinical and administrative information. By combining scheduling, prescriptions, laboratory workflows, pharmacy inventory, billing, and notifications in a single platform, the system makes hospital operations more efficient and easier to use for both patients and staff, while providing a scalable foundation for future enhancement.

Chapter 2

Related Works

Hospital information systems have evolved significantly with the adoption of digital platforms that manage patient records, scheduling, pharmacy, and billing. Modern systems provide electronic health records, appointment management, secure access, and reporting. While these platforms have transformed healthcare delivery, many existing solutions either focus on large enterprise deployments or require complex configuration unsuitable for small institutions. This chapter reviews existing hospital-management platforms related to the proposed HMS and identifies the gaps it aims to address.

2.1 Review of Existing Works

OpenMRS

OpenMRS is a widely used open-source medical-record platform designed to support healthcare delivery, particularly in resource-constrained settings. OpenMRS (2026)

- **Technologies / Methodology:** Java-based web platform with a configurable data model for patients, encounters, and observations, extended through a modular architecture.
- **Strengths:** Centralised patient record, role-based access, active global community, and high configurability.
- **Limitations:** Requires a dedicated server environment and technical expertise to install, configure, and maintain, which is a barrier for small institutions without dedicated IT staff.

While OpenMRS provides a comprehensive medical-record platform, the proposed HMS focuses on a simpler, integrated solution that combines appointments, pharmacy, and billing in one role-based system tailored for small and medium-sized hospitals.

OpenEMR

OpenEMR is a mature open-source electronic health record and practice-management system used by clinics and hospitals worldwide. OpenEMR (2026)

- **Technologies / Methodology:** PHP and MySQL web application supporting scheduling, prescriptions, billing, and reporting.
- **Strengths:** Feature-rich, certified for clinical use, and supported by an established community.
- **Limitations:** Its breadth brings configuration complexity, and its older architecture is less suited to lightweight deployment or full-stack learning environments.

Compared to OpenEMR, the proposed HMS delivers the core functionalities required for efficient hospital management using a modern, modular full-stack architecture that is simpler to deploy and understand.

HospitalRun

HospitalRun is an open-source hospital information system aimed at small and rural hospitals, with an emphasis on offline-first operation. HospitalRun (2026)

- **Technologies / Methodology:** JavaScript-based platform covering patient records, scheduling, billing, and inventory, designed to work in low-connectivity environments.
- **Strengths:** Usability focus and reliable operation where connectivity is limited.
- **Limitations:** Feature set and active maintenance vary across releases, and role separation is limited compared with enterprise systems.

Unlike HospitalRun, the proposed HMS provides fine-grained role-based access across five distinct roles and integrates pharmacy inventory, billing, and multi-channel notifications within a single, actively structured codebase.

Chapter 3

Design and Implementation

The Hospital Management System is a role-based web platform developed to connect patients and hospital staff through a single digital system. It provides role-specific dashboards that allow patients to book appointments and view records, and staff to manage schedules, prescriptions, laboratory work, pharmacy stock, and billing.

3.1 System Overview

The HMS addresses the need for a simple, integrated hospital platform by bringing appointments, clinical records, pharmacy, billing, and notifications together in one secure, role-based system. Each role interacts only with the features appropriate to it, and all data flows through a single backend over a well-defined RESTful API.

3.1.1 Major Components

- **Frontend Development:** The web interface is developed using React.js with the Vite build tool. Tailwind CSS is used for styling, React Router for client-side navigation, and Axios as the API client for communicating with the backend.
- **Backend Development:** Node.js with the Express.js framework is used for server-side development. The backend follows a REST API architecture for client–server communication, with MySQL as the relational database accessed through the Sequelize Object–Relational Mapping (ORM) library. JSON Web Tokens secure the API, and Nodemailer provides outbound email.
- **Development Tools:** Visual Studio Code (VS Code) is used as the primary code editor. Git and GitHub are used for version control and source-code management, Nodemon for automatic server reloading during development, and ESLint for code quality.

3.1.2 Component Interaction

- Patients and staff interact with the React web application to perform role-specific actions such as booking appointments, issuing prescriptions, processing lab tests, managing pharmacy stock, and settling invoices.
- The frontend communicates with the backend through RESTful APIs over HTTP, under the common base path `/api/v1/hms`. The backend processes requests, performs business logic, and returns JSON responses.
- The backend authenticates users using JWT-based authentication and authorises access based on user roles before performing CRUD operations on the MySQL database through Sequelize.
- Access tokens are short-lived and refreshed through rotating refresh tokens stored in HTTP-only cookies, which can be revoked to end all sessions.
- External services, such as the email (SMTP) and SMS providers, are invoked by the backend to send appointment, prescription, billing, and account notifications to users.

3.2 System Requirement Specifications

3.2.1 Software Requirements

Frontend Development

- **Programming Language:** JavaScript (ES2022)
- **Frameworks / Libraries:** React.js, Vite
- **Styling:** Tailwind CSS
- **API Client:** Axios

Backend Development

- **Programming Language:** JavaScript (Node.js)
- **Framework:** Express.js
- **API Architecture:** Representational State Transfer (REST) API
- **Database:** MySQL with the Sequelize ORM
- **Authentication / Email:** JSON Web Tokens, bcrypt, Nodemailer

Development Tools

- **Code Editor:** Visual Studio Code (VS Code)
- **Version Control:** Git, GitHub
- **Runtime Tooling:** Nodemon, ESLint
- **Operating System:** Windows

3.2.2 Functional Requirements

- Users shall be able to register, log in, and manage their accounts securely, with password reset and email verification.
- Patients shall be able to book, view, and cancel appointments based on doctor availability.
- Doctors shall be able to issue prescriptions, order laboratory tests, and refer patients to other doctors.
- Lab assistants shall be able to process test requests and record results, which patients can then view.
- Administrators shall be able to manage staff accounts, rooms, and billing.
- Pharmacists shall be able to manage a location-aware medicine inventory with stock levels and reorder thresholds.
- The system shall create in-app notifications and deliver them by email.

3.2.3 Non-Functional Requirements

- The system shall provide a responsive and user-friendly interface across major web browsers.
- The system shall support multiple concurrent users while maintaining consistent performance.
- The system shall ensure secure authentication and protect medical data from unauthorised access.
- The system shall be scalable and maintainable to accommodate future features.

3.3 System Design

3.3.1 Architectural Design

The HMS follows a layered client–server architecture. The presentation layer (React SPA) renders role-specific dashboards and calls the backend over REST; the application layer (Express) enforces authentication, authorisation, and business logic through controllers, middleware, and services; and the data layer (MySQL via Sequelize) stores all persistent records with enforced relationships. Figure 3.1 shows the overall architecture.

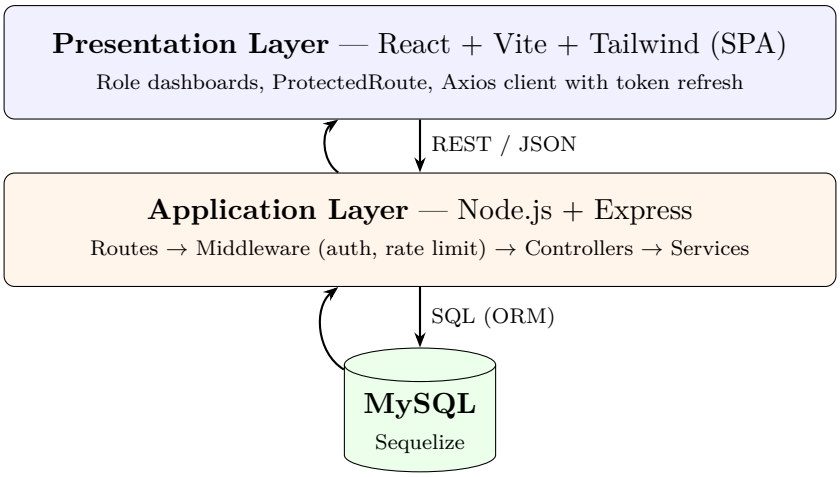

Figure 3.1: System architecture

3.3.2 Database Design

The HMS uses MySQL as its primary database to store application data. The schema is normalised into related tables for the key entities, including **Users**, **Patients**, **Staff Profiles**, **Appointments**, **Prescriptions**, **Lab Tests**, **Referrals**, **Invoices**, **Medicines**, and **Notifications**. Relationships between tables are enforced using foreign keys with cascade rules, allowing efficient and consistent data retrieval. This schema provides the integrity and scalability required to support scheduling, clinical records, billing, and pharmacy management. Figure 3.2 shows the entity–relationship overview.

3.4 Implementation Details

This section describes how the proposed architecture was implemented. The system was developed using a layered client–server architecture consisting of a React web application and an Express backend, integrated through RESTful APIs to provide a seamless hospital-management experience.

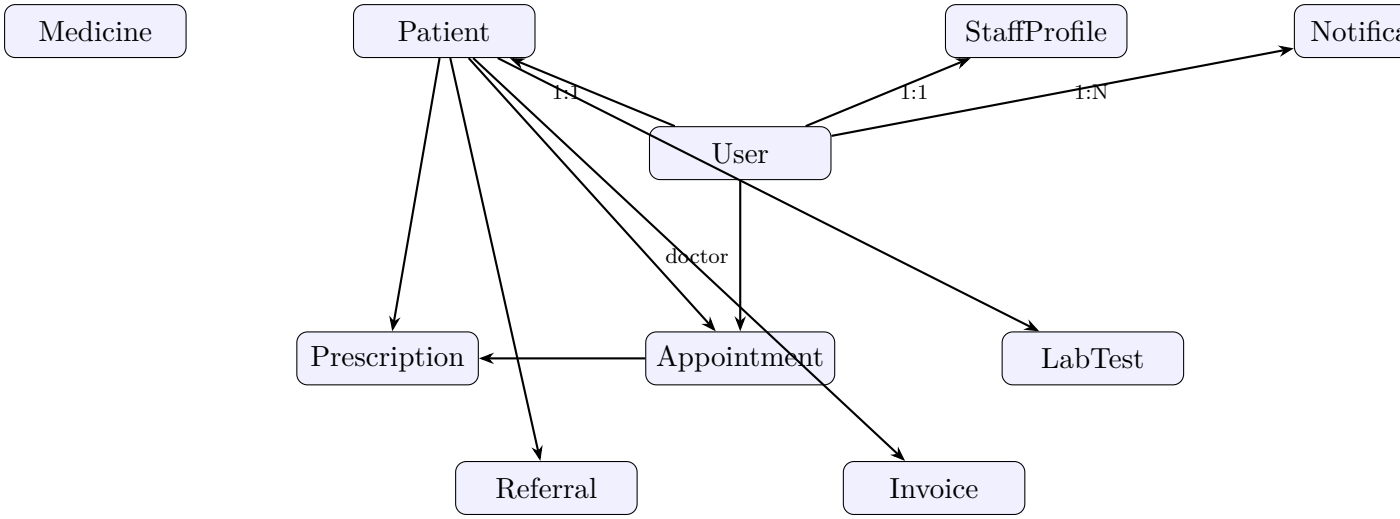

Figure 3.2: ER diagram (principal entities and relationships)

3.4.1 Frontend Implementation

The frontend is a Single Page Application developed using React.js, Vite, and Tailwind CSS. It provides role-specific dashboards for patients, doctors, pharmacists, lab assistants, and administrators. A `ProtectedRoute` component restricts each dashboard to its allowed roles, authentication state is held in a React context, and Axios attaches the JWT access token to every request while transparently refreshing it on expiry. Toast notifications provide user feedback.

3.4.2 Backend Implementation

The backend was developed using Node.js, Express.js, and the Sequelize ORM in a modular structure with separate routes, controllers, middleware, and services. Controllers handle authentication, appointments, availability, prescriptions, lab tests, referrals, invoices, notifications, medicines, and staff. Middleware enforces JWT authentication, role-based authorisation, and rate limiting on sensitive endpoints, while MySQL stores all persistent data. Password hashes use bcrypt, and reset/verification tokens are stored only as SHA-256 hashes.

3.4.3 Integration of Components

The frontend communicates with the backend through RESTful APIs over HTTP. The backend interacts with MySQL through Sequelize for persistent storage and enforces referential integrity with foreign-key relationships. External services, including the email (SMTP) and SMS providers, are integrated with the backend to send notifications; when these are not configured, the system falls back to console logging so all flows remain runnable in development. This modular implementation keeps the system scalable, main-

tainable, and easy to extend.

3.5 Flowcharts

This section presents the flowcharts illustrating the workflow and system logic of the HMS. They describe the sequence of operations for user registration, authentication, appointment booking, and role-based access, showing how users and the backend interact to complete operations within the system.

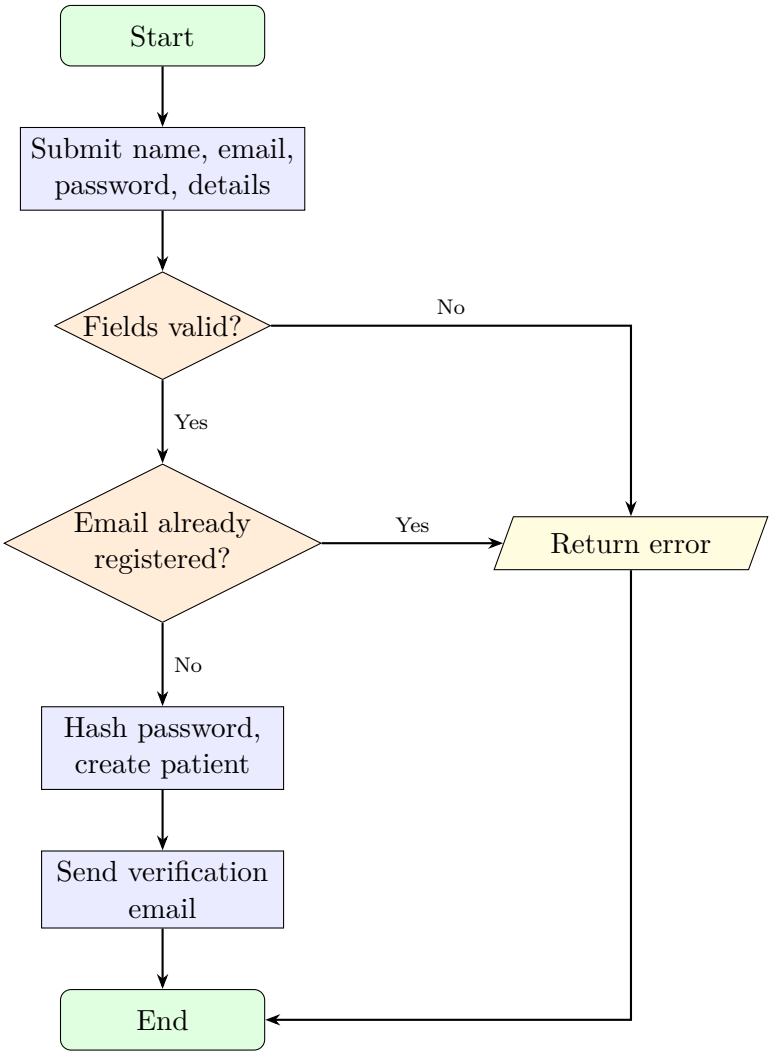

Figure 3.3: Registration flowchart

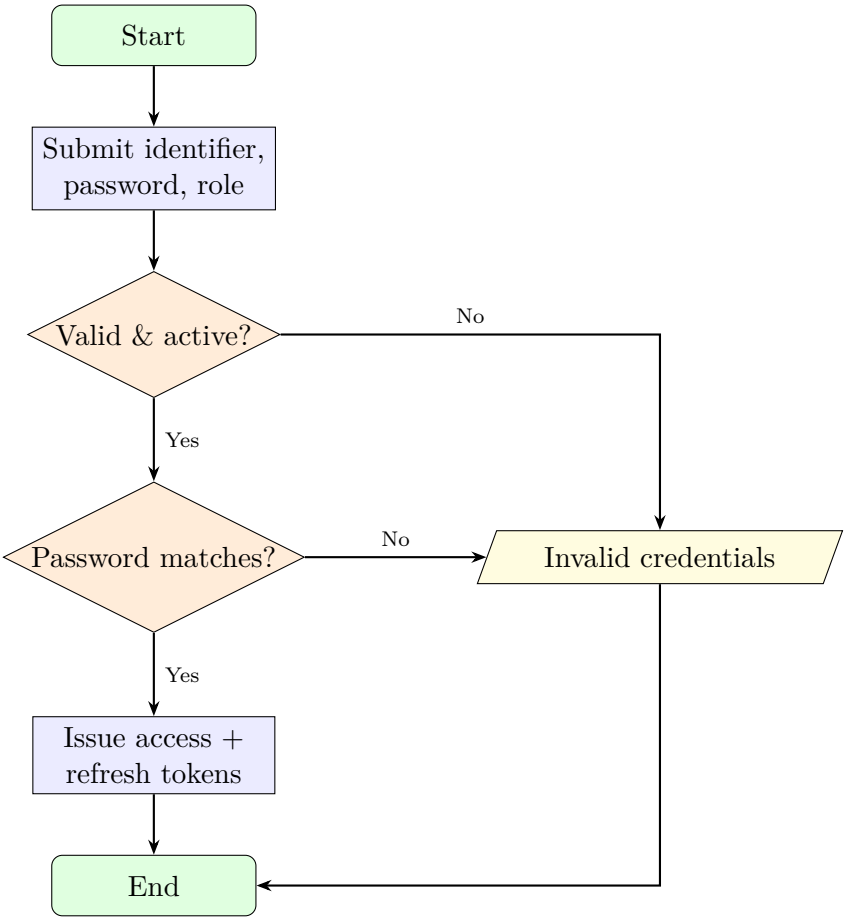

Figure 3.4: Login flowchart

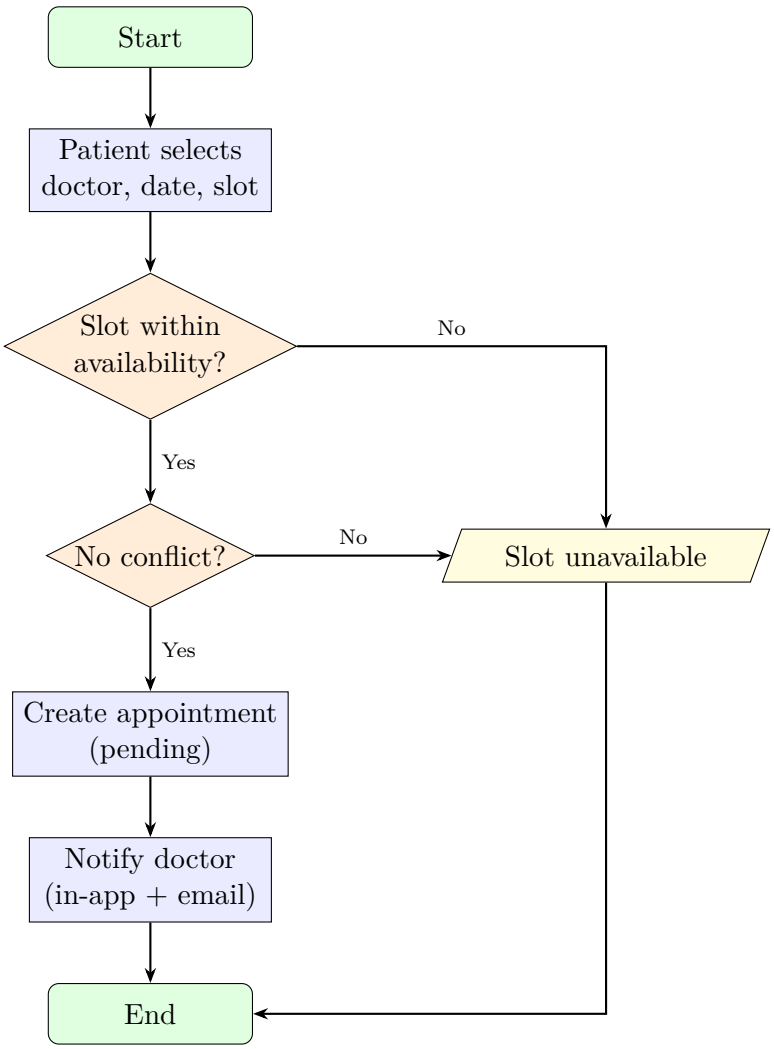

Figure 3.5: Appointment booking flowchart

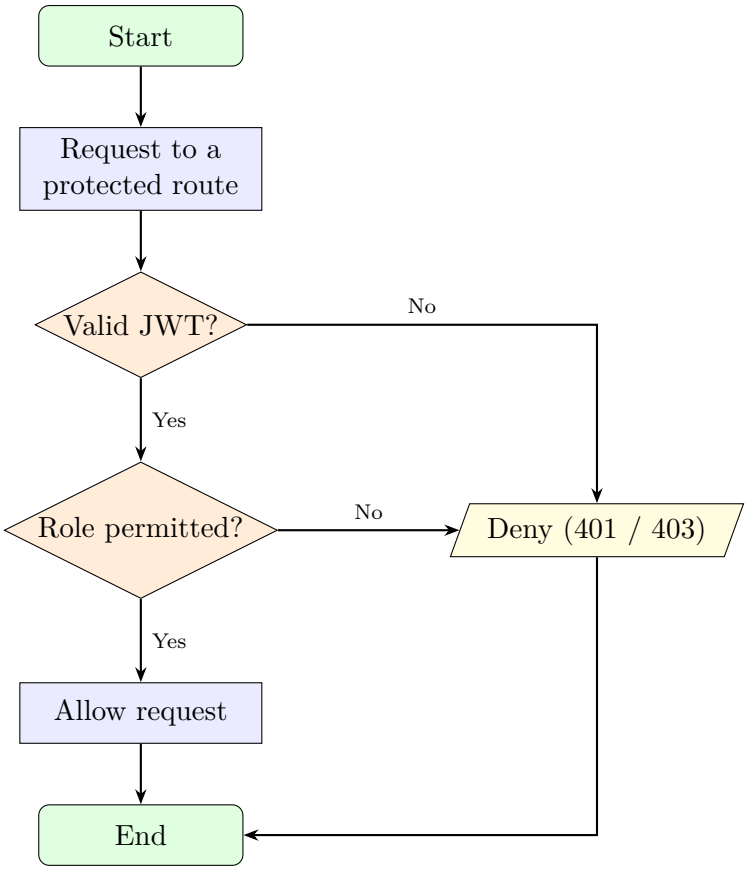

Figure 3.6: Role-based access control flowchart

Chapter 4

Results and Discussion

4.1 Implemented Features

The Hospital Management System successfully implemented several core functionalities:

- **User Authentication:** Users register and log in using JWT-based authentication with refresh-token rotation, together with password reset and email verification, ensuring protected access to authorised resources.
- **Role-Based Dashboards:** Each of the five roles (patient, doctor, pharmacist, lab assistant, administrator) has a distinct dashboard exposing only its permitted features.
- **Appointment Management:** Patients book appointments against a doctor's declared availability with conflict checking, and appointments follow a pending/confirmed/cancelled/co workflow.
- **Clinical Records:** Doctors issue prescriptions, order laboratory tests, and refer patients; lab assistants process test requests and record results.
- **Billing and Pharmacy:** Administrators issue invoices and record payments, while pharmacists manage a location-aware medicine catalogue with stock levels, reorder thresholds, and expiry.
- **Notifications:** In-app notifications are created for key events and mirrored to the user's email, keeping users informed without logging in.

Figures A.1–A.8 in Appendix A show the running system.

4.2 Results and Performance Analysis

The implemented system was tested to evaluate its functionality, performance, and reliability. Testing confirmed that the core features operated correctly:

- **Authentication:** JWT-based authentication allowed secure registration, login, and access to protected resources while preventing unauthorised access; password reset revoked existing sessions correctly.
- **Appointment and Clinical Records:** Appointments, prescriptions, lab tests, and referrals were created and updated successfully, with records stored and retrieved accurately from MySQL.
- **Billing and Pharmacy:** Invoices and payments were recorded with correct status transitions, and the medicine catalogue displayed accurate stock and location data.
- **Database Performance:** MySQL handled CRUD operations for users, appointments, and invoices with quick response times under normal load.
- **System Stability:** The backend started, connected to the database, synchronised all models, and served requests without major errors; the frontend built and ran without runtime errors.

4.3 Comparison with Objectives or Existing Systems

The implemented system successfully achieves its primary objectives by providing an integrated platform for hospital management through a role-based web application. During analysis and design, existing systems such as OpenMRS, OpenEMR, and HospitalRun were studied to understand their features and workflows. Unlike these platforms—which offer broad enterprise features but require significant configuration—the proposed system focuses on the core functionalities required for efficient hospital management: secure authentication, appointments, prescriptions, laboratory workflows, referrals, billing, and pharmacy inventory, within a simple and maintainable full-stack codebase.

Some deviations from the original plan:

- **Managed migrations:** the schema is auto-synchronised in development rather than versioned through migrations.
- **Transactions:** some multi-step flows (such as recording payments) are not yet fully transactional.
- **SMS delivery:** notifications are delivered in-app and by email; SMS is supported but disabled unless a provider is configured.
- **Inpatient module:** ward and bed management were considered but reserved for future development.

4.4 Discussion

The implementation demonstrates the effectiveness of modern web technologies in building a unified hospital-management platform. By combining a React frontend, an Express backend, and a MySQL database accessed through Sequelize, the system provides a consistent and responsive experience. REST APIs enable clean communication between the frontend and backend, while JWT-based authentication with refresh-token rotation ensures secure, revocable access. The layered, modular architecture improves maintainability and allows new features to be added with minimal changes. The normalised relational schema efficiently manages relationships among patients, staff, appointments, and billing, keeping the system scalable for future expansion such as an inpatient module, audit logging, and analytics.

Chapter 5

Conclusion and Future Works

The Hospital Management System was successfully designed and implemented using modern web development technologies to provide a secure, role-based platform for patients and hospital staff. The system adopts a layered client–server architecture, with React powering the web application and Node.js with Express handling backend services. MySQL, accessed through Sequelize, serves as the primary database, offering a reliable and scalable solution for storing clinical and administrative records. Secure authentication is achieved through JWT with refresh-token rotation, while communication between the frontend and backend uses REST APIs. The implementation provides essential features such as appointment scheduling, prescriptions, laboratory workflows, referrals, billing, and pharmacy inventory, with notifications delivered in-app and by email. Overall, the proposed system is efficient, reliable, scalable, and user-friendly, providing a solid foundation for future enhancements.

5.1 Limitations

Although the proposed system implements the core functionalities required for hospital management, it has several limitations that can be addressed in future versions.

- The database schema is auto-synchronised in development rather than managed through versioned migrations.
- Some multi-step operations are not yet fully transactional.
- Real email and SMS delivery depend on external providers being configured; otherwise messages fall back to console logging.
- The system is web-only, with no dedicated mobile application.
- An automated test suite is not yet in place; verification is currently manual.
- Inpatient/ward management and audit logging are not yet implemented.

5.2 Future Enhancements

Several improvements can be incorporated into future versions:

- Adopt versioned database migrations and wrap critical flows (payments, booking) in transactions.
- Add an automated test suite with continuous integration.
- Implement an inpatient/ward and bed-management module.
- Introduce audit logging of access to medical data to support privacy and compliance.
- Add automatic pharmacy stock deduction on dispensing and low-stock alerts.
- Develop a companion mobile application for patients.
- Expand analytics with dashboards tracking hospital usage and trends.

Bibliography

[1] OpenMRS (2026). *OpenMRS.* https://openmrs.org/. Accessed: July 26, 2026.

[2] OpenEMR (2026). *OpenEMR.* https://www.open-emr.org/. Accessed: July 26, 2026.

[3] HospitalRun (2026). *HospitalRun.* https://hospitalrun.io/. Accessed: July 26, 2026.

[4] React (2026). *React documentation.* https://react.dev/. Accessed: July 26, 2026.

[5] Node.js (2026). *Node.js documentation.* https://nodejs.org/en/docs/. Accessed: July 26, 2026.

[6] Sequelize (2026). *Sequelize ORM documentation.* https://sequelize.org/. Accessed: July 26, 2026.

[7] MySQL (2026). *MySQL documentation.* https://dev.mysql.com/doc/. Accessed: July 26, 2026.

Appendix A

Additional Figures and Screenshots

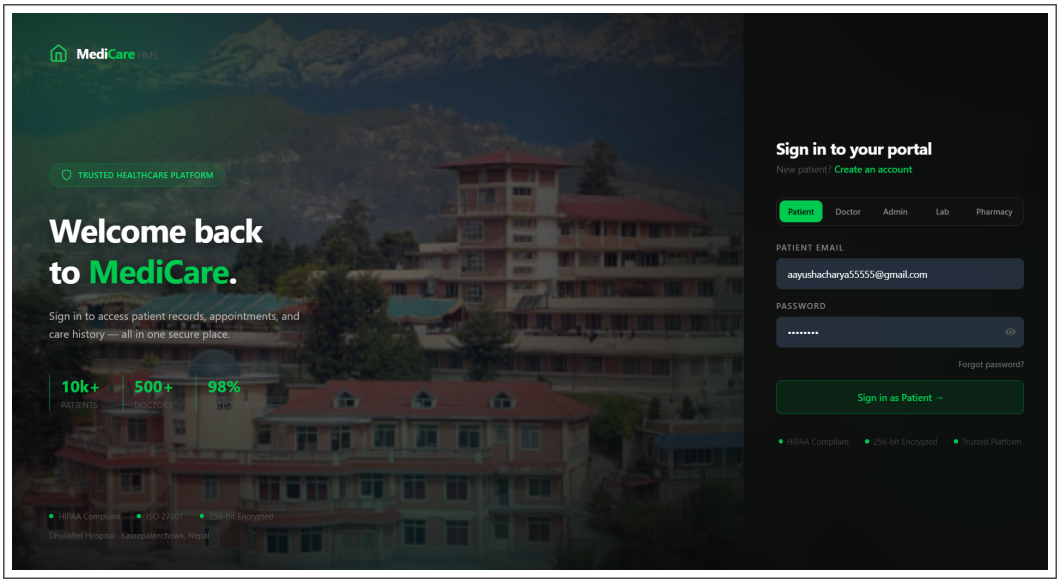

Figure A.1: Login

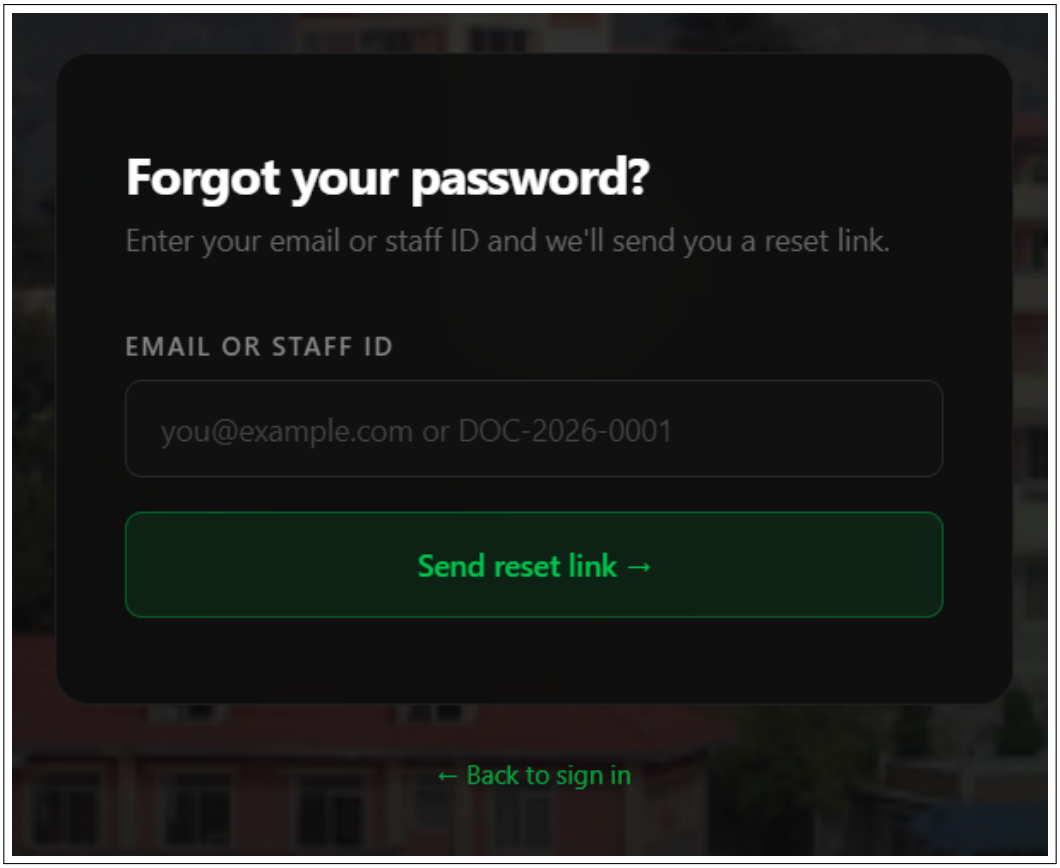

Figure A.2: Forgot password

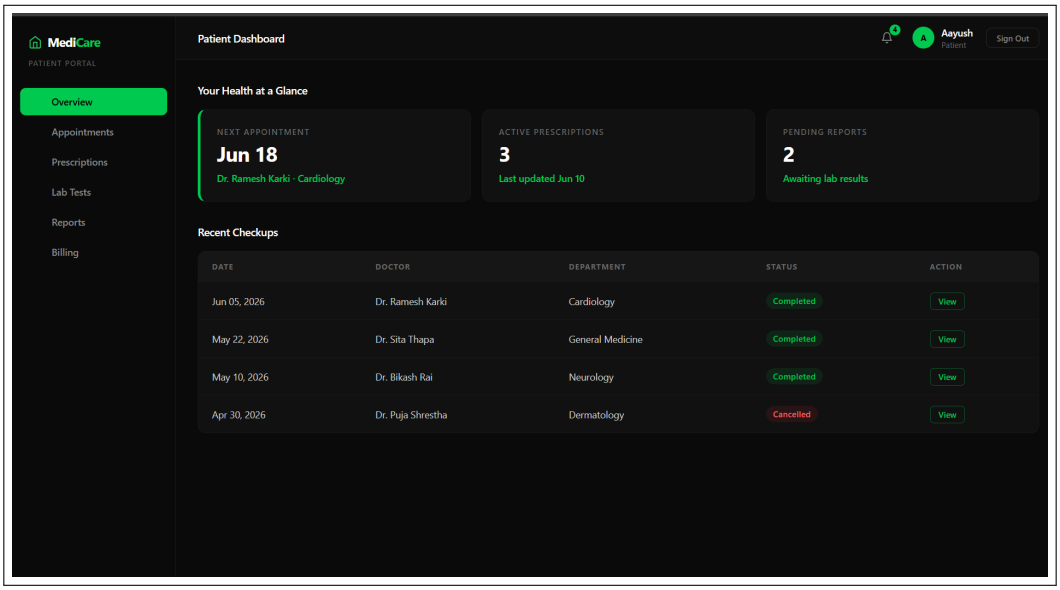

Figure A.3: Patient dashboard

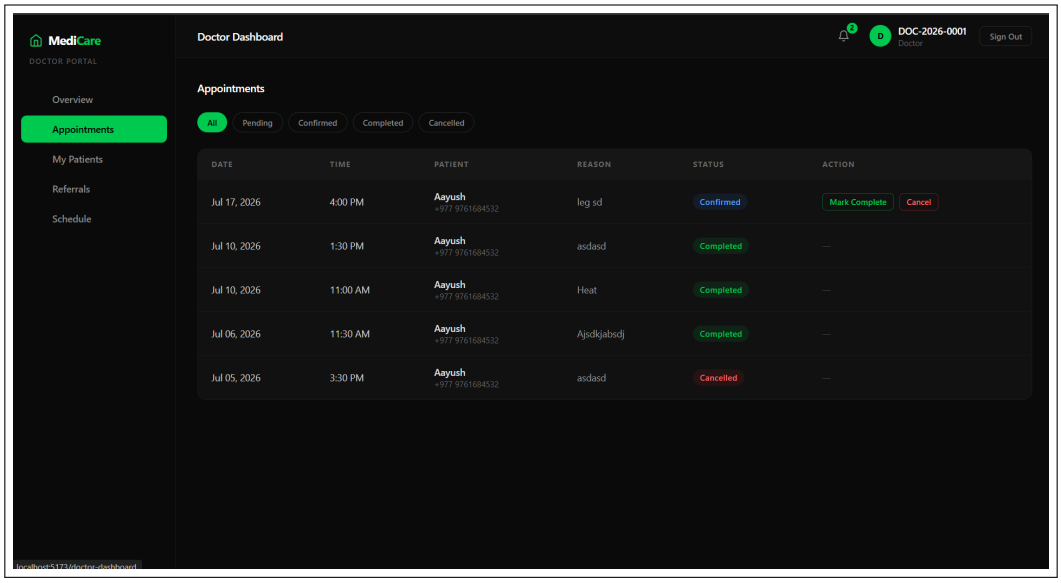

Figure A.4: Doctor dashboard

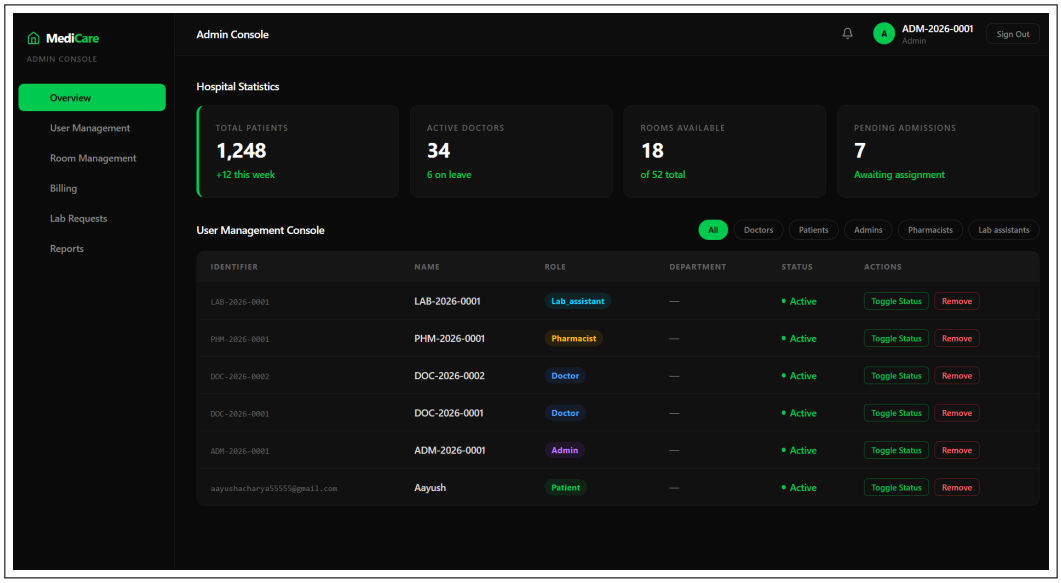

Figure A.5: Admin dashboard

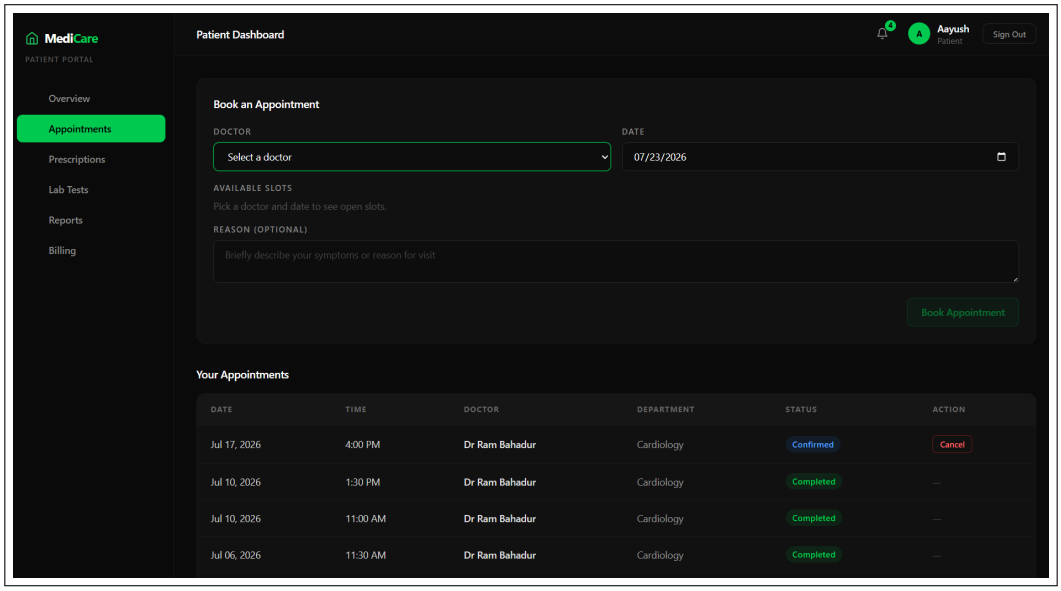

Figure A.6: Appointment booking and management

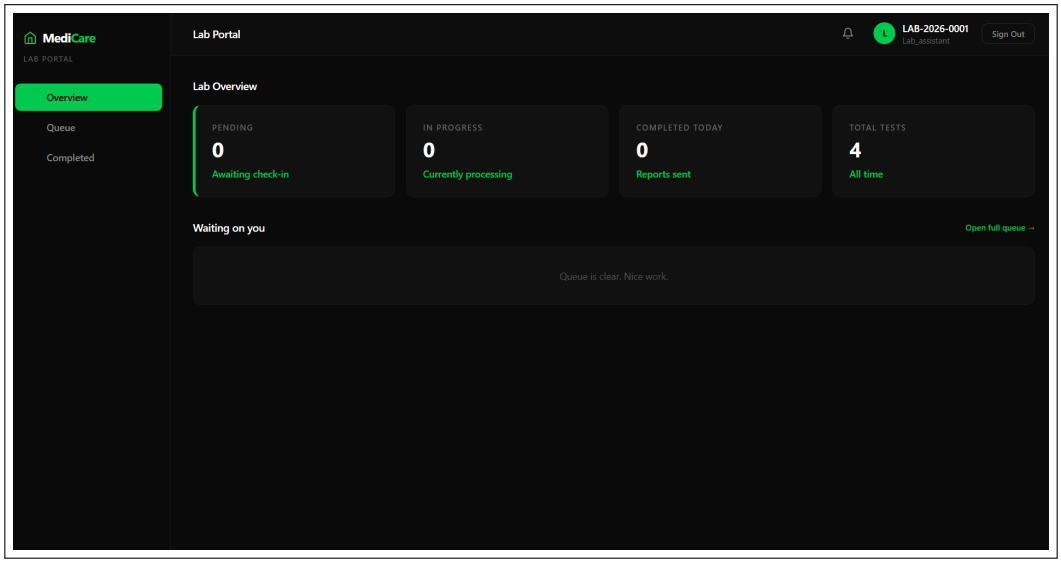

Figure A.7: Lab Assistant portal

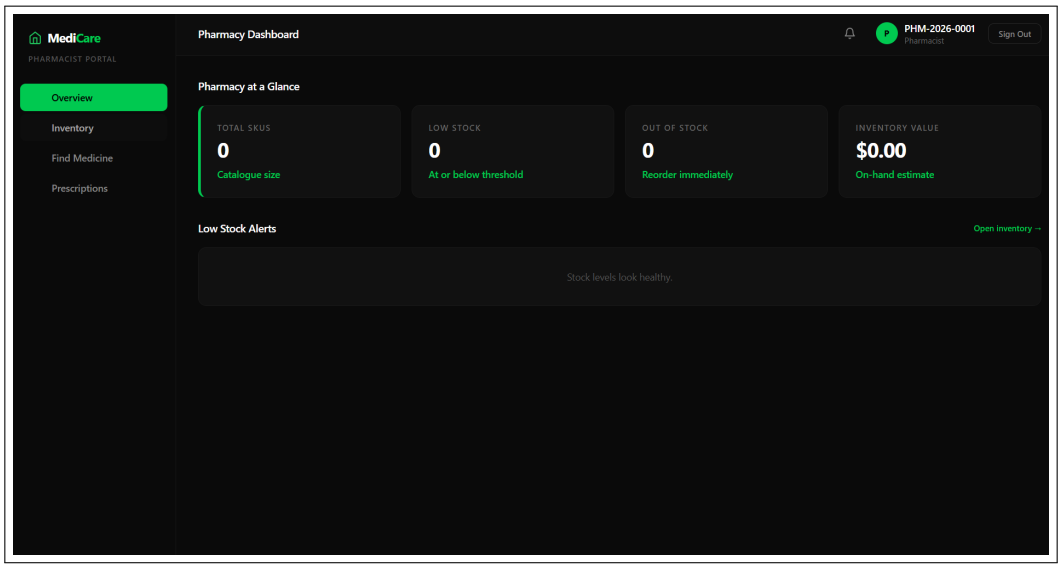

Figure A.8: Pharmacist portal — medicine inventory